

Like and Comment Functionality Implementation

Overview

This implementation adds social interaction features to community posts, including:

- **Like functionality:** Users can like/unlike posts with a toggle mechanism
- **Comment functionality:** Users can add, edit, and delete their own comments
- **Persistent storage:** All interactions are stored in the PostgreSQL database
- **User authentication:** Integrated with existing NextAuth authentication system

Changes Made

1. Database Schema Updates (`app/prisma/schema.prisma`)

Added two new models:

PostLike Model

- Stores user likes on posts
- Unique constraint on `userId` and `postId` to prevent duplicate likes
- Indexed for efficient queries

PostComment Model

- Stores user comments on posts
- Users can edit and delete their own comments
- Timestamps for `createdAt` and `updatedAt`
- Indexed for efficient queries

2. API Routes Created

`/api/community/posts/[postId]/like/route.ts`

- **POST:** Toggle like/unlike on a post
- **GET:** Get like status and count for a post
- Returns current like state and total like count

`/api/community/posts/[postId]/comments/route.ts`

- **GET:** Fetch all comments for a post with user details
- **POST:** Add a new comment to a post
- Returns comment with user information and ownership status

`/api/community/posts/[postId]/comments/[commentId]/route.ts`

- **PUT:** Update a comment (only by the comment owner)
- **DELETE:** Delete a comment (only by the comment owner)
- Enforces ownership validation

3. Updated API Routes

`/api/community/posts/route.ts`

Enhanced to include:

- `likeCount` : Total number of likes on each post
- `commentCount` : Total number of comments on each post
- `isLiked` : Whether the current user has liked the post

4. Frontend Component Updates

`app/components/community/community-overview.tsx`

Complete redesign with:

New Features:

- Interactive like button with visual feedback (filled heart when liked)
- Comment section with expandable/collapsible UI
- Add comment form with textarea and send button
- Comment list with author names and timestamps
- Edit and delete buttons for own comments (with ownership validation)
- Loading states for comments
- Toast notifications for all actions
- Smooth animations using Framer Motion

State Management:

- `expandedComments` : Tracks which posts have comments expanded
- `comments` : Stores comments for each post
- `newComment` : Manages new comment input state
- `editingComment` : Tracks which comment is being edited
- `loadingComments` : Shows loading state per post

User Experience:

- Real-time like count updates
- Optimistic UI updates for better responsiveness
- Clear visual indicators for liked posts
- Inline editing for comments
- Confirmation dialog before deleting comments
- Timestamps showing when comments were created/edited

Database Migration

Applying the Migration

The migration SQL file is located at:

```
app/prisma/migrations/add_like_comment_functionality.sql
```

To apply the migration, you have two options:

Option 1: Using Prisma Migrate (Recommended)

```
cd app
npx prisma migrate dev --name add_like_comment_functionality
```

Option 2: Manual SQL Execution

Execute the SQL file directly against your PostgreSQL database:

```
psql $DATABASE_URL -f prisma/migrations/add_like_comment_functionality.sql
```

After migration, regenerate the Prisma client:

```
npx prisma generate
```

Testing

Manual Testing Steps

1. Start the development server:

```
bash
cd app
npm run dev
```

2. Test Like Functionality:

- Navigate to the community page (/community)
- Click the like button on a post
- Verify the count increases and heart fills with color
- Click again to unlike
- Verify the count decreases and heart becomes outline

3. Test Comment Functionality:

- Click on “Comments” to expand the comment section
- Add a new comment using the textarea
- Verify the comment appears with your name and timestamp
- Edit your comment using the edit icon
- Delete your comment using the trash icon
- Verify you cannot edit/delete other users’ comments

4. Test Persistence:

- Refresh the page
- Verify likes and comments are still there
- Test with multiple users to verify ownership rules

API Endpoints Reference

Like Endpoints

- POST /api/community/posts/[postId]/like - Toggle like
- GET /api/community/posts/[postId]/like - Get like status

Comment Endpoints

- GET /api/community/posts/[postId]/comments - List comments
- POST /api/community/posts/[postId]/comments - Add comment
- PUT /api/community/posts/[postId]/comments/[commentId] - Update comment
- DELETE /api/community/posts/[postId]/comments/[commentId] - Delete comment

Security Features

1. **Authentication:** All endpoints require valid NextAuth session
2. **Authorization:** Users can only edit/delete their own comments
3. **Validation:** Content validation prevents empty comments
4. **SQL Injection Protection:** Using Prisma ORM with parameterized queries
5. **Unique Constraints:** Prevents duplicate likes from same user

Performance Optimizations

1. **Indexed Queries:** Database indexes on frequently queried fields
2. **Lazy Loading:** Comments are only loaded when expanded
3. **Optimistic Updates:** UI updates immediately for better UX
4. **Efficient Queries:** Using Promise.all for parallel data fetching
5. **Unique Constraint:** Prevents duplicate data at database level

Future Enhancements (Optional)

- Nested replies/threading for comments
- Notifications when someone comments on your post
- Like animation effects
- Comment pagination for posts with many comments
- Sort comments by newest/oldest
- Rich text editor for comments
- @ mentions in comments
- Like list showing who liked a post

Dependencies

No new dependencies were added. The implementation uses existing packages:

- `@prisma/client` - Database ORM
- `next-auth` - Authentication
- `framer-motion` - Animations (already installed)
- `sonner` - Toast notifications (already installed)
- Radix UI components (already installed)

Rollback Instructions

If needed, to rollback the changes:

1. Drop the new tables:

```
DROP TABLE IF EXISTS "PostComment";  
DROP TABLE IF EXISTS "PostLike";
```

1. Revert the code changes by checking out the previous commit:

```
git checkout main
```

Notes

- The implementation treats exercise submissions as posts (using submission ID as post ID)
- Comments and likes are stored with both `postId` and `submissionId` for flexibility
- The UI matches the existing vintage/literary theme of the application
- All error cases are handled with appropriate toast notifications
- The code follows existing patterns in the codebase